

4 Тестирование веб-приложения

4.1 Функциональное тестирование

Для проверки корректности работы всех функций разработанного веб-приложения было проведено ручное тестирование с использованием инструмента Postman [68] и пользовательского интерфейса веб-приложения. Описание и итоги тестирования представлены в таблице 4.1.

Тестирование сквозных функций, таких как регистрации и аутентификации, представлено в таблице 4.1.

Таблица 4.1 – Описание тестирования функций регистрации и аутентификации

Номер	Функция веб-приложения	Описание тестирования	Ожидаемый результат	Полученный результат
1	2	3	4	5
1	Регистрация	Отправить POST запрос на адрес /sso/signUp, указав в теле запроса имя пользователя (name) со значением «test», адрес электронной почты (email) со значением «test@gmail.com», пароль (password) со значением «12345678»	В таблице user должна появиться запись со следующими значениями для колонок: колонка id с значением «уникальный uuid», колонка email с значением «test@gmail.com», колонка role со значением «client», колонка пароль со значением «хэш от пароля», колонка photo со значением «null», колонка name со значением «test», колонка created_time со значением приблизительного времени создания, колонка updated_time со значением приблизительного времени создания. Сервер должен вернуть ответ с кодом 200, данные пользователя, access token, access token expiration в формате JSON. Установить в куки refresh token и refresh token expiration	Совпадает с ожидаемым.

					ДП 04.00.ПЗ			
		Ф.И.О.	Подпись	Дата				
Разраб.	Козляковский Д.А				4 Тестирование веб-приложения	Лит.	Лист	Листов
Провер.	Сухорукова И. Г.					У	1	8
Н. контр.	Сухорукова И. Г.							
Утверд.	Смелов В.В.					БГТУ 1-40 01 01, 2025		

Продолжение таблицы 4.1

1	2	3	4	5
2	Аутентификация	Отправить POST запрос на адрес /sso/signIn, указав в теле запроса имя пользователя (name) со значением «Danila», адрес электронной почты (email) со значением «danilakozlyakovksy@gmail.com», пароль (password) со значением «TankiDanik2003»	Сервер должен вернуть ответ с кодом 200, данные пользователя, access token, access token expiration в формате JSON. Установить в куки refresh token и refresh token expiration.	Совпадает с ожидаемым.

Тестирование общих функций, доступных всем пользователям, представлено в таблице 4.2.

Таблица 4.2 – Описание тестирования общих функций.

Номер	Функция веб-приложения	Описание тестирования	Ожидаемый результат	Полученный результат
3	Просмотр информации о абоне-ментах	Отправить GET запрос на адрес /abonements, указав в заголовке Authorization значение – Bearer и значение валидного access токена.	Сервер должен вернуть ответ с кодом 200 и список абонементов.	Совпадает с ожидаемым.
4	Поиска, сортировка, фильтрация абоне-ментов	На ui перейти на вкладку абонементов, нажав на кнопку ABONEMENTS на навигационной панели слева. Использовать элементы для сортировки, фильтрации, поиска.	В списке абонементов отображаются нужные карточки абонементов, в правильной последовательности.	Совпадает с ожидаемым.
5	Просмотр информации о тренерах и отзывов о тренерах	Отправить GET запрос на адрес /coaches, указав в заголовке Authorization значение – Bearer и значение валидного access токена.	Сервер должен вернуть ответ с кодом 200 и список абонементов с их услугами.	Совпадает с ожидаемым.

Продолжение таблицы 4.2

1	2	3	4	5
6	Просмотр расписания тренера	Отправить GET запрос на адрес /training/day/:day/coach/:coachId указав в URL дату и идентификатор тренера. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для клиента, администратора или тренера.	Сервер должен вернуть ответ с кодом 200, в теле ответа должен быть список объектов тренировок.	Совпадает с ожидаемым.

Тестирование функций, доступных пользователям с ролью «Клиент», представлено в таблице 4.3.

Таблица 4.3 – Описание тестирования функций пользователей с ролью «Клиент».

Номер	Функция веб-приложения	Описание тестирования	Ожидаемый результат	Полученный результат
7	Публикация отзыва о тренере	Отправить POST запрос на адрес /reviews, указав в теле запроса (UserId) со значением существующего id пользователя, (Body) со значением валидного тела отзыва, (CoachId) со значением существующего id тренера. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для клиента.	В таблице review должна появиться запись со следующими значениями для колонок: колонка id с значением «уникальный uuid», колонка body со значением «norm coach», колонка user_id со значением id пользователя, передаваемого в теле запроса, колонка created_time со значением приблизительного времени создания, колонка updated_time со значением приблизительного времени создания. В таблице coach_review должна появиться запись отзыва. Сервер должен вернуть ответ с кодом 200, в теле ответа должен быть объект отзыва.	Совпадает с ожидаемым.

Продолжение таблицы 4.3

1	2	3	4	5
8	Редактирование профиля	Отправить PUT запрос на адрес /users/:id, указав в URL id пользователя, в теле запроса параметры: (name) со значением «test_change», (photo) со значением фото в бинарном формате. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для клиента.	В таблице user должна обновиться запись пользователя с id передаваемым в URL запроса. Должны обновиться колонки: name со значением «test_change», photo со значением URL фото из localstack(s3), колонкой updated_time со значением приблизительного времени обновления. В localstack(s3) должна появиться добавиться фото из тела запроса. Сервер должен вернуть ответ с кодом 200, в теле ответа должен быть объект обновленного пользователя.	Совпадает с ожидаемым.

Тестирование функций, доступных пользователям с ролью «Администратор», представлено в таблице 4.4.

Таблица 4.4 – Описание тестирования функций пользователей с ролью «Администратор».

Номер	Функция веб-приложения	Описание тестирования	Ожидаемый результат	Полученный результат
9	Удаление клиентов	Отправить DELETE запрос на адрес /users/:id, указав в URL id пользователя. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для администратора.	Из таблицы user должна удалиться запись клиента с id переданным в URL запроса. Из localstack(s3) должна удалиться фото удаляемого клиента. Сервер должен вернуть ответ с кодом 200, в теле ответа должен быть объект удаленного клиента.	Совпадает с ожидаемым.

Продолжение таблицы 4.4

1	2	3	4	5
10	Создание абонемента	Отправить POST запрос на адрес /abonements, в теле запроса с content type = multipart/form-data указать параметры: (title) со значением «test», (validity_period) со значением «1», (visiting_time) со значением «7.00 – 14.00», (price) со значением «100», (services) со значением id абонементов перечисленных через запятую, (photo) со значением фото в бинарном формате. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для админа.	В таблице abonement должна появиться запись со следующими значениями для колонок: колонка id с значением «уникальный uuid», колонка title с значением «test», колонка validity со значением «1», колонка visiting_time с значением «7.00 - 14.00», колонка photo со значением URL фото из localstack(s3), колонка price со значением «100», колонка created_time со значением приблизительного времени создания, колонка updated_time со значением приблизительного времени создания. В аккаунте Stripe должны создаваться объекты продукта с названием «test» и цены со значением «100». В localstack(s3) должно добавиться фото абонемента. Сервер должен вернуть ответ с кодом 200, данные абонемента.	Совпадает с ожидаемым.
11	Удаление отзыва к тренеру	Отправить DELETE запрос на адрес /reviews/:id, указав в URL id отзыва. Указать в заголовке Authorization значение – Bearer и значение валидного access токена.	Из таблицы taining должна удалиться запись тренировки с id переданным в URL запроса. Сервер должен вернуть ответ с кодом 200, в теле ответа должен быть объект удаленного отзыва.	Совпадает с ожидаемым.

Продолжение таблицы 4.4

1	2	3	4	5
12	Обновление абонемента	Отправить PUT запрос на адрес /abonements, в теле запроса с content type = multipart/form-data указать параметры: (title) со значением «test_updated», (validity_period) со значением «3», (visiting_time) со значением «14.00 – 24.00», (price) со значением «200», (services) со значением id абонементов перечисленных через запятую, (photo) со значением фото в бинарном формате. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для админа.	В таблице abonement должна обновиться запись со следующими значениями для колонок: колонка id с значением «уникальный uuid», колонка title с значением «test_updated», колонка validity со значением «3», колонка visiting_time с значением «14.00 – 24.00», колонка photo со значением обновленного URL фото из localstack(s3), колонка price со значением «200», колонка updated_time со значением приблизительного времени обновления. В аккаунте Stripe должен создаться объект цены со значением «200» и присвоится к продукту, также должно обновиться имя продукта. В localstack(s3) должно добавиться фото абонемента. Сервер должен вернуть ответ с кодом 200, данные обновленного абонемента.	Совпадает с ожидаемым.
13	Создание тренера	Отправить POST запрос на адрес /coach, в теле запроса с content type = multipart/form-data указать параметры: (name) со значением «test», (description) со значением «some description to coach», (services) со значением id услуг перечисленных через запятую, (photo) со значением фото в бинарном формате. Указать в заголовке Authorization значение – Bearer access токена.	В таблице coach должна появиться запись со следующими значениями для колонок: колонка id с значением «уникальный uuid», колонка name с значением «test», колонка description с значением «some description to coach» колонка photo со значением URL фото из s3, колонка created_time со значением времени создания, колонка updated_time со значением времени создания. В s3 должно добавиться фото абонемента.	Совпадает с ожидаемым.

Продолжение таблицы 4.4

1	2	3	4	5
14	Обновление тренера	Отправить PUT запрос на адрес /coach, в теле запроса с content type = multipart/form-data указать параметры: (name) со значением «testUp», (description) со значением «some description to coachUp», (services) со значением id услуг перечисленных через запятую, (photo) со значением фото в бинарном формате. Указать в заголовке Authorization значение – Bearer и значение валидного access токена.	В таблице coach должна обновиться запись со следующими значениями для колонок: колонка name с значением «testUp», колонка description с значением «some description to coachUp» колонка photo со значением URL фото из localstack(s3), колонка updated_time со значением приблизительного времени обновления. В localstack(s3) должно обновиться фото абонемента. Сервер должен вернуть ответ с кодом 200, данные обновленного тренера.	Совпадает с ожидаемым.
15	Удаление абонемента	Отправить DELETE запрос на адрес /coach/:id, указав в URL id тренера. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для администратора.	Из таблицы coach должна удалиться запись тренера с id переданным в URL запроса. Из localstack(s3) должна удалиться фото удаляемого тренера. Сервер должен вернуть ответ с кодом 200, в теле ответа должен быть объект удаленного тренера.	Совпадает с ожидаемым.
16	Бронирование онлайн тренировки	Отправить POST запрос на адрес /training/book указав в теле идентификатор тренера, время начала тренировки, время окончания тренировки, дату тренировки. Указать в заголовке Authorization значение – Bearer и значение access токена.	В таблицу training должна добавиться запись с полями id, time_from, time_until, status, coach_id, client_id, created_time, updated_time. Сервер должен вернуть ответ с кодом 200, в теле ответа должен быть объект удаленной тренировки.	Совпадает с ожидаемым.
17	Отмена бронирования онлайн тренировки	Отправить DELETE запрос на адрес /training/:id указав в URL id тренировки. Указать в заголовке Authorization значение – Bearer и значение валидного access токена.	Из таблицы training должна удалиться запись с переданным id. Сервер должен вернуть ответ с кодом 200.	Совпадает с ожидаемым.

Завершение таблицы 4.4

1	2	3	4	5
18	Вхождение в комнату онлайн тренировки	Отправить GET запрос на адрес /training/join/:roomIdcoach/:coachId указав в URL идентификатор комнаты. Указать в заголовке Authorization значение – Bearer и значение валидного access токена для клиента или тренера.	Сервер должен установить websocket соединение с клиентом. Вставить объект websocket подключения в хранилище на уровне приложения. Запустить процесс обмена данными для установления WebRTC соединения. Должен вернуть ответ с кодом 101.	Совпадает с ожидаемым.

Тестирование оплаты и списка заказов представлены в таблице 4.5.

Таблица 4.5. Тестирование оплаты и списка своих заказов через клиентский браузер

Шаг	Действие	Ожидаемый результат	Результат
1	Перейти на страницу абонементов, нажав на кнопку АБОНЕМЕНТЫ на навигационной панели слева.	Карточки абонементов отображаются на странице	Совпадает с ожидаемым
2	Инициация процесса оплаты: нажать на кнопку «КУПИТЬ».	Сервер генерирует сессию Stripe и отправляет URL со страницы оплаты stripe на клиент. Происходит перенаправление на страницу оформления платежа	Совпадает с ожидаемым
3	Ввод тестовых данных на странице оплаты.	Платёж проходит через Stripe, после чего пользователя перенаправляют на главную. История заказов доступна в личном профиле.	Совпадает с ожидаемым
4	Просмотр истории заказов: после нажатия на кнопку «ПРОФИЛЬ» пользователь видит список своих заказов.	История заказов отображается на странице профиля	Совпадает с ожидаемым

Таким образом, были протестированы все ключевые функции веб-приложения.

4.2 Выводы по разделу

Проведено тестирование ключевых функций веб-приложения через Postman и UI — все результаты соответствовали ожидаемым. Выполнено 19 позитивных тестов, что составляет 100% покрытия без учета негативного тестирования. Проверена оплата через Stripe в браузере Microsoft Edge [69].